

Modern Cloud Computing

Lecture 4: Cloud Storage

Chongzhou Fang

Rochester Institute of Technology

Today's Topic

- Basic Concepts
- Network File Systems
- Distributed File Systems
- Google's Storage Solutions
- Storage System Design Principles

Cloud Storage

Cloud Computing: computing + storing, moving, and processing massive data.

Examples:

- Sensors and mobile devices continuously generate data
- Cloud services collect detailed operational and user data
- Scientific applications process terabytes or petabytes of data
- Large-scale analytics depends on colocating storage and processing

Big Data: Three Dimensions

Big Data growth as a three-dimensional phenomenon:

1. Volume

- More data is generated and stored

2. Velocity

- Data must be processed quickly to produce useful results

3. Diversity

- Data comes from many sources and formats

Examples:

- Mobile photos and videos
- Sensor streams
- ...

Cloud Storage: Challenges

Cloud storage must provide:

- Large capacity
- Low access latency
- High throughput
- High availability
- Low cost
- Consistency
- Failure recovery

But these goals often conflict.

Example: Replication improves availability, but makes consistency harder.

Reliability at the Lowest Possible Cost

Older distributed file systems often assumed more reliable components.

Modern cloud storage systems are built from:

- Commodity disks
- Commodity servers
- Commodity networks
- Frequent component failures

The design philosophy shifted from:

| Performance at any cost

To:

| Reliability at the lowest possible cost

HDD vs. SSD

Feature	HDD	SSD
Storage medium	Magnetic platters	Flash memory / integrated circuits
Moving parts	Yes	No
Access latency	Higher	Lower
Shock resistance	Lower	Higher
Cost per byte	Lower	Higher
Typical role	Bulk storage	Fast storage / cache / high IOPS

SSD: reduces latency

HDD: remains attractive for massive storage because of cost.

Storage Models

A **storage model** describes how data structures are laid out in physical storage.

Two abstract models:

1. Cell storage

- Storage is divided into equal-size cells
- Each object fits in one cell

2. Journal storage

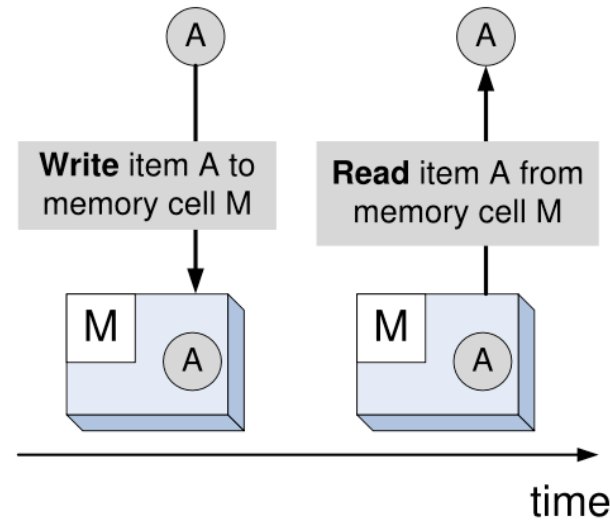
- Maintains the history of updates
- Uses logs to support recovery and atomicity

Cell Storage: Desired Properties

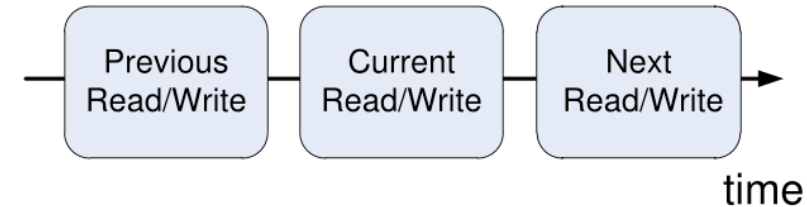
Two key correctness properties:

Read/write coherence

- A read should return the result of the most recent write to that cell



Read/Write coherence: the result of a **Read** of memory cell M should be the same as the most recent **Write** to that cell



Before-or-after atomicity: the result of every **Read** or **Write** is the same as if that **Read** or **Write** occurred either completely before or completely after any other **Read** or **Write**.

Before-or-after atomicity

- Each read or write appears to happen completely before or completely after another operation

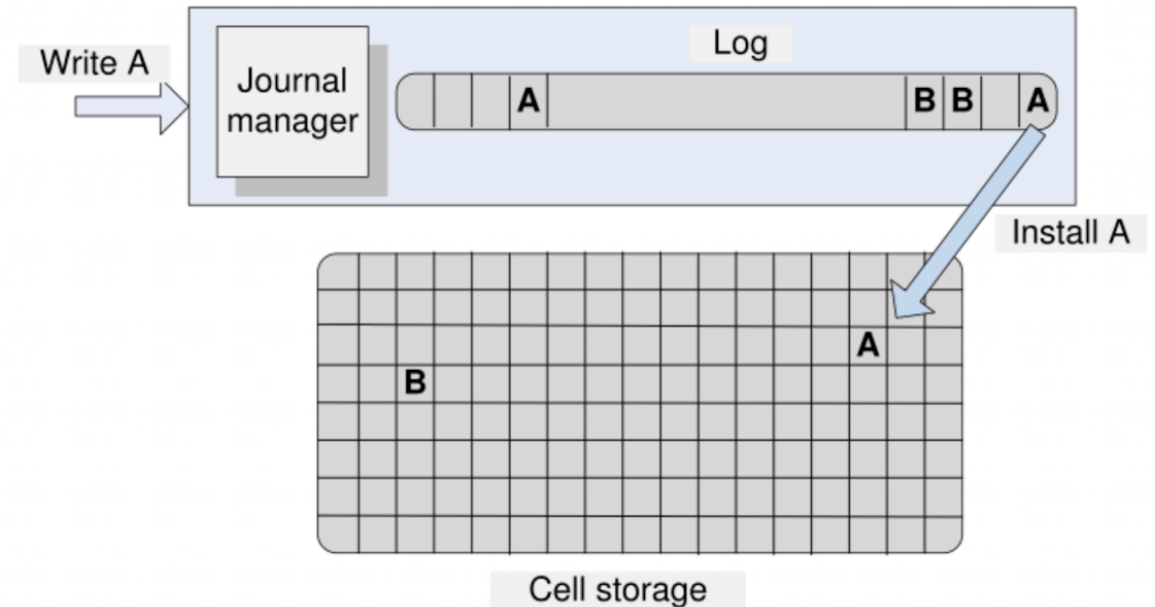
Journal Storage and Logs

Journal storage maintains history of all variables.

The log records updates before installing them into cell storage.

This supports **all-or-nothing** actions:

- If the system fails before the log is complete, do not update the cell storage
- If the system fails after the log is complete, recover using the log



Databases and DBMS

Most cloud applications do not interact directly with file systems.

They use databases through an application layer.

A DBMS provides:

- Data integrity
- Data access management
- Concurrency control
- Recovery after failure
- Query language support

Cloud challenge: low latency + scalability + availability + consistent view of data.

Cloud Database Tradeoff

Traditional relational databases are easy to use and support strong transaction semantics.

But many cloud workloads need:

- Massive scale
- Very high availability
- Low latency
- Flexible or semi-structured data
- High write/read throughput

This motivates NoSQL, BigTable-like systems, and hybrid systems such as Megastore.

Distributed File Systems

Distributed File Systems: Motivation

Early distributed file systems were motivated by networks of workstations.

Goals included:

- Share files across machines
- Reduce administration effort
- Support diskless workstations
- Provide transparent remote access
- Preserve familiar file-system semantics

Examples:

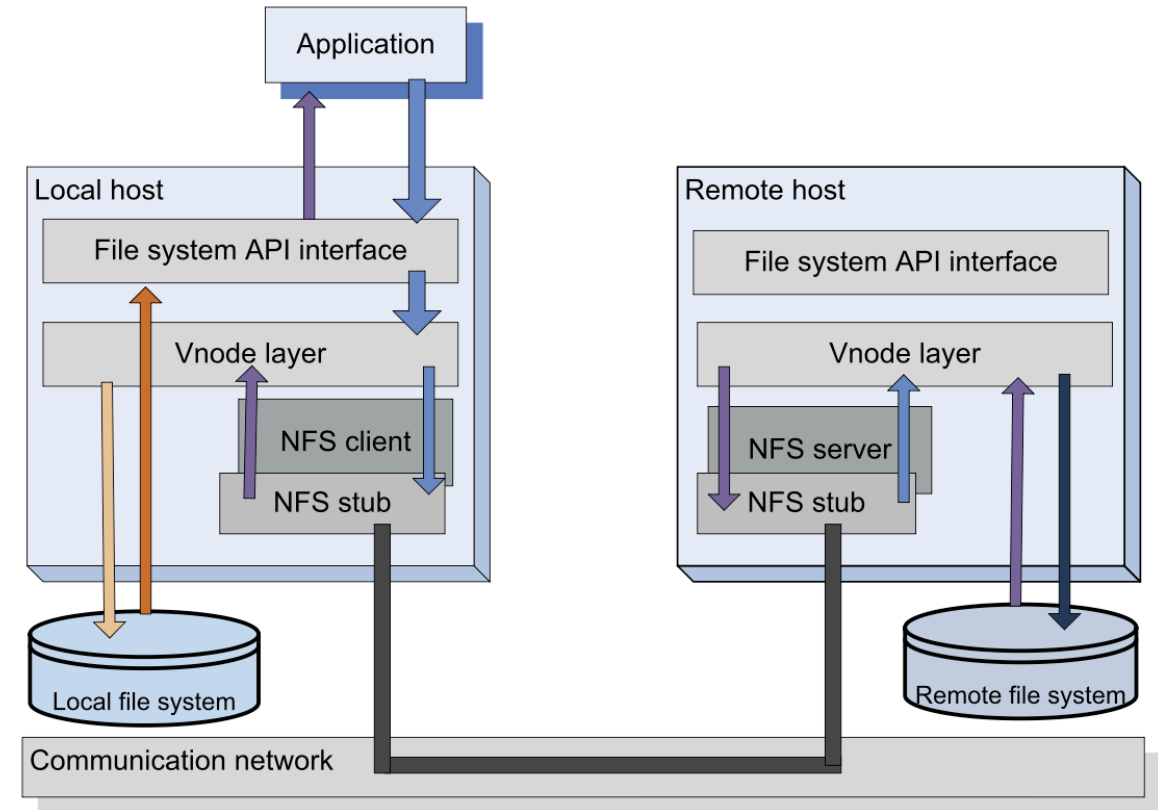
- NFS
- AFS
- Sprite File System

NFS: Network File System

NFS is based on the client-server model.

- Client runs on the local host
- Server manages the remote file system
- Communication happens through RPCs
- The vnode layer hides whether a file is local or remote

Important limitation: the NFS server can become a bottleneck or single point of failure.



UFS Layering and NFS Design

UNIX File System ideas influenced NFS.

UFS separates:

- Logical file structure
- Physical file structure
- Metadata through inodes
- Paths and names from disk blocks

This layered design made it easier to extend local file access to remote file access.

File Descriptor vs. File Handle

A local file is identified by a **file descriptor**.

A remote NFS file is identified by a **file handle**.

A file handle includes information such as:

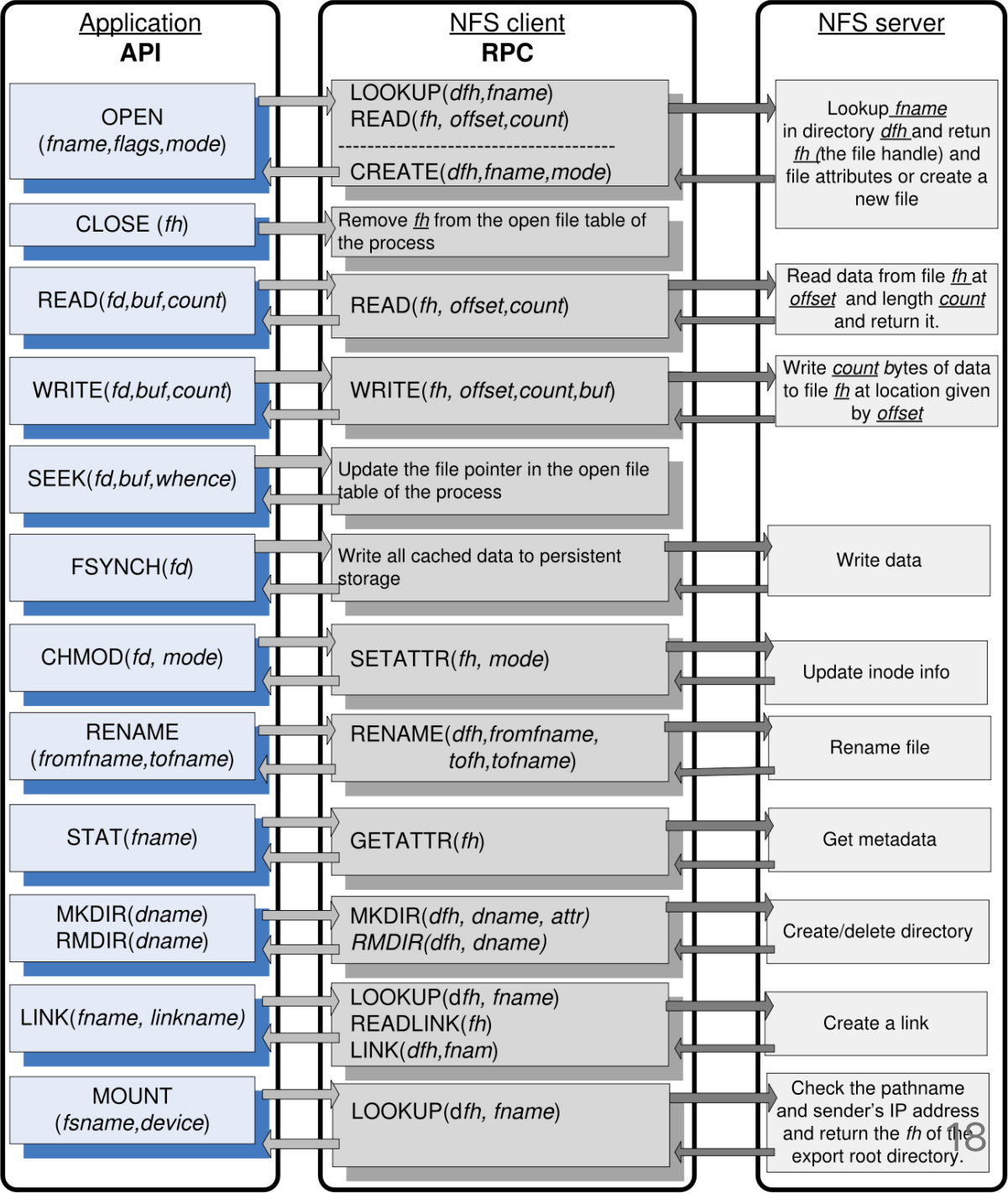
- File system identifier
- Inode number
 - inode: a data structure in Unix-like file systems that stores all metadata and block pointers for a file or directory
- Generation number
 - The generation number helps avoid incorrect reuse of inode numbers when multiple clients operate on the same remote file.

NFS Operations

NFS maps local file-system API calls to remote procedure calls.

Examples:

Local operation	NFS/RPC idea
OPEN	lookup file handle
READ	read by file handle and offset
WRITE	write by file handle and offset
STAT	get metadata
MKDIR/RMDIR	create/delete directory



AFS: Andrew File System

AFS was designed for a very large number of workstations.

Important ideas:

- Trusted servers form **Vice**
- Workstations use a process called **Venus**
- Local disk acts as a persistent cache
- Modified files are stored back to servers
- Access Control List (ACL) supports sharing
 - Specifies access rights of users

Sprite File System

Sprite File System used caching aggressively.

A file request could be served from:

1. Local client cache
2. Local file system
3. Remote server cache
4. Remote server disk

Sprite supported both client and server caching and used consistency mechanisms to preserve expected semantics.

Comparing Early Network File Systems

Comparison

Table 6.2 A comparison of several network file systems [353].				
File system	Cache size and location	Writing policy	Consistency guarantees	Cache validation
NFS	Fixed, memory	On close or 30 sec. delay	Sequential	On open, with server consent
AFS	Fixed, disk	On close	Sequential	When modified server asks client
SFS	Variable, memory	30 sec. delay	Sequential, concurrent	On open, with server consent
Locus	Fixed, memory	On close	Sequential, concurrent	On open, with server consent
Apollo	Variable, memory	Delayed or on unlock	Sequential	On open, with server consent
RFS	Fixed, memory	Write-through	Sequential, concurrent	On open, with server consent

Parallel File Systems

A distributed file system shares files across machines.

A **parallel file system** allows many clients to read and write concurrently from the same file.

This is important for:

- Scientific computing
- Large-scale simulations
- Data-intensive analytics
- High-performance clusters

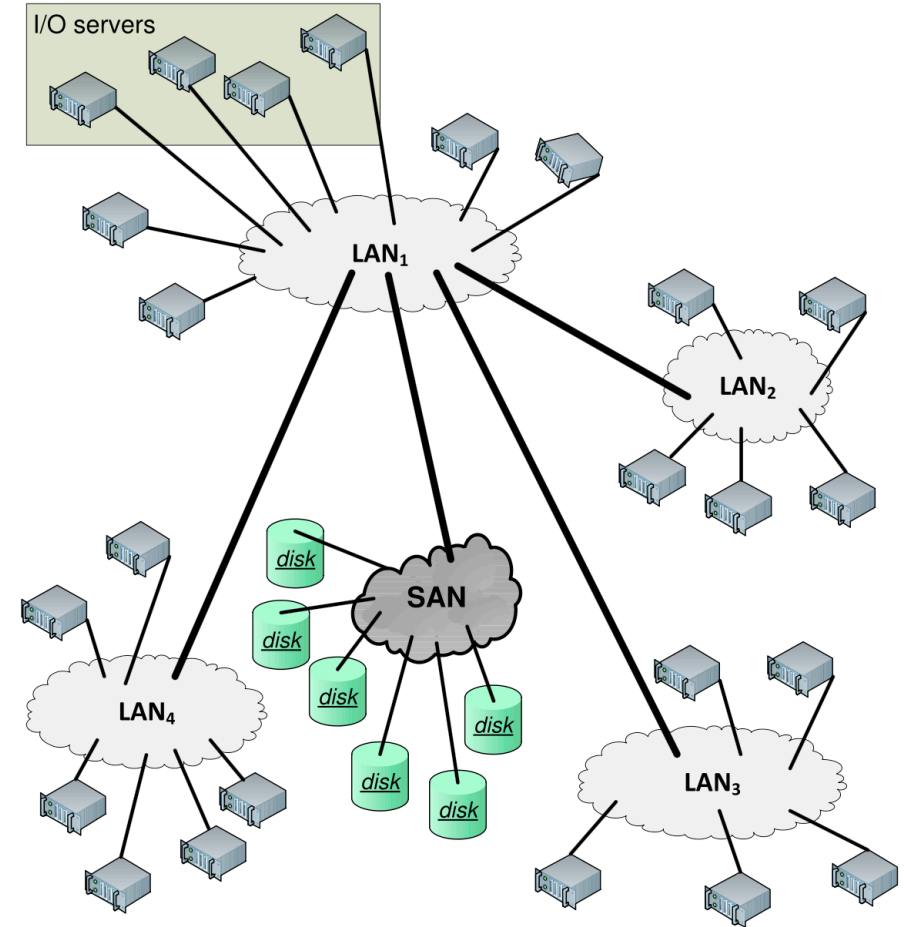
Challenge: improve I/O throughput while preserving consistency.

GPFS: General Parallel File System

GPFS was developed by IBM for large clusters.

Features:

- Parallel file system emulating behavior of a general-purpose POSIX system running on a single system
- Files striped across multiple disks
- Large file-system capacity
- Parallel I/O
- Metadata support
- Recovery through write-ahead logs
- Distributed locking



GPFS: Striping and Recovery

GPFS stripes files across disks to improve performance.

But striping has a downside:

- A single disk failure may affect many files

Reliability mechanisms:

- RAID devices
- Replication of data and metadata
- Write-ahead log for metadata updates
 - Write-ahead: updates are written to persistent storage only after log records have been written
- Any I/O node can initiate recovery

GPFS: Consistency Through Distributed Locking

Concurrent access improves performance but complicates consistency.

GPFS uses:

- A central lock manager
- Local lock managers on I/O nodes
- Lock tokens
- Byte-range tokens for read/write operations
- Metanodes for metadata updates

Lock granularity affects performance.

■ Fine-grained locking improves concurrency but increases coordination overhead.

Google's Storage Solutions

Google File System

Google File System was designed for:

- Petabytes of storage
- Thousands of commodity machines
- Large user population
- Frequent failures
- Large files
- Bulk data processing

The main concern was reliability under hardware and software failures.

GFS Workload Assumptions

GFS was designed around observed workload characteristics:

- Files are often GBs to hundreds of TBs
- Sequential reads are common
- Appends are common
- Random writes are rare
- Users process data in bulk
- Throughput matters more than latency

Storage systems should be designed for the workload they actually serve.

GFS Design Decisions

GFS design decisions:

- Split files into large chunks (fixed-size segments)
- Support atomic append
- Use high-bandwidth networking
- Avoid client-side caching (avoid consistency overhead)
- Use a master for critical metadata operations
- Minimize master involvement in common file access
- Support checkpointing and garbage collection

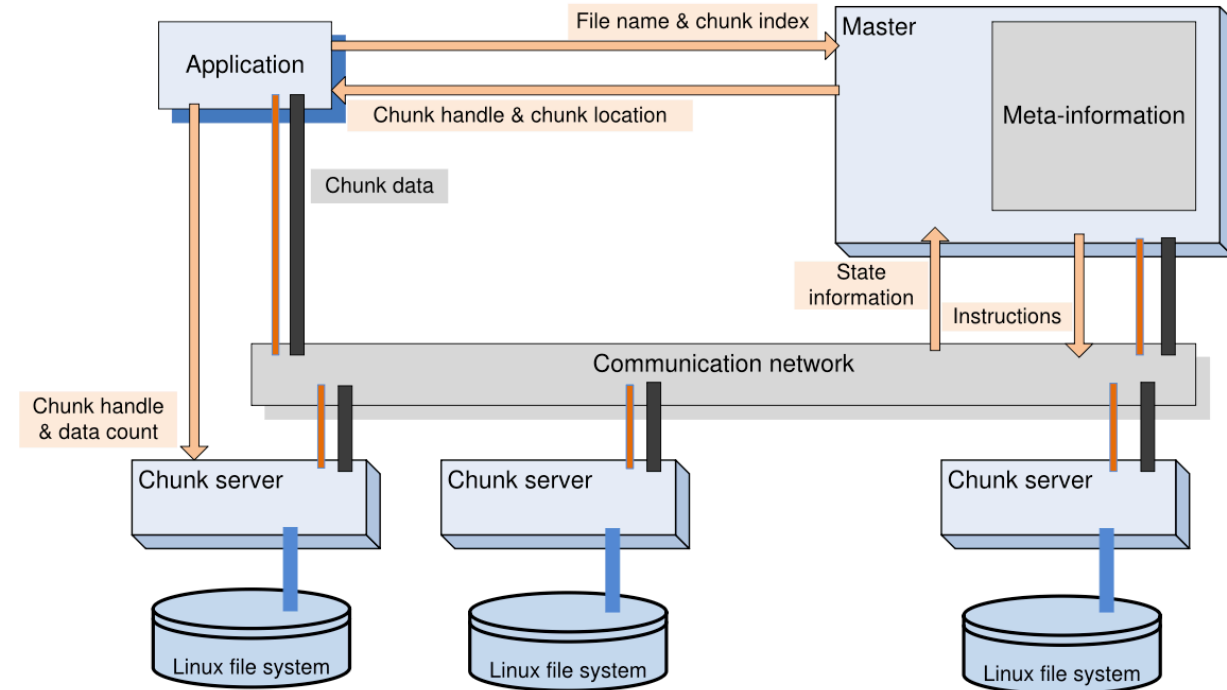
GFS Architecture

A GFS cluster contains:

- One master
- Many chunk servers
- Many clients/applications

The master maintains:

- File names
- Access control information
- File-to-chunk mapping
- Chunk replica



GFS Chunks

GFS files are divided into chunks.

Important details:

- Each chunk has a unique chunk handle
- Chunk size is 64 MB
- Each chunk contains 64 KB blocks
- Each block has a checksum
- Chunks are replicated across multiple chunk servers
- Default number of replicas is three

Large chunks reduce metadata and help high-throughput sequential access.

GFS Write Flow

Simplified write process:

1. Client asks master for chunk replica locations
2. Master identifies primary and secondary chunk servers
3. Client sends data to all replicas
4. Client sends write request to the primary
5. Primary orders mutations using sequence numbers
6. Secondaries apply mutations in the same order
7. Primary replies to the client after acknowledgments

GFS Reliability Mechanisms

GFS uses several reliability mechanisms:

- Replication
- Operation log
- Master checkpointing
- Checksums for chunks
- Heartbeats between master and chunk servers
- Lazy garbage collection
- Copy-on-write snapshots

Example: Deleted files are first renamed to hidden names, giving users a recovery window.

Locks

Locks help distributed storage systems enforce controlled access to shared data.

They support:

- Atomic reads/writes
- Master election
- Metadata consistency
- Coordination among replicas
- Recovery protocols

Cloud storage systems need locks because components can fail and messages can be delayed, lost, or reordered.

Advisory vs. Mandatory Locks

Lock type	Meaning	Tradeoff
Advisory lock	Processes voluntarily follow the locking rules	Flexible but can be bypassed
Mandatory lock	System blocks access if lock is not held	Stronger control but higher overhead and less flexibility

Advisory, coarse-grained locks are often better for very large distributed systems.

Fine-Grained vs. Coarse-Grained Locks

Lock type	Benefit	Cost
Fine-grained	More concurrency, being held shorter	Higher lock-server workload
Coarse-grained	Lower coordination cost, being held longer	Less concurrency

For large-scale systems, coarse-grained locks reduce the load on the lock service and reduce sensitivity to lock-server failures.

Chubby Lock Service

Chubby is Google's distributed lock service.

It is used by systems such as:

- GFS
- BigTable

Design choices:

- Advisory locks
- Coarse-grained locks
- Paxos-based consensus
- Replicated service
- Client API similar to a simplified file system

Chubby Cell

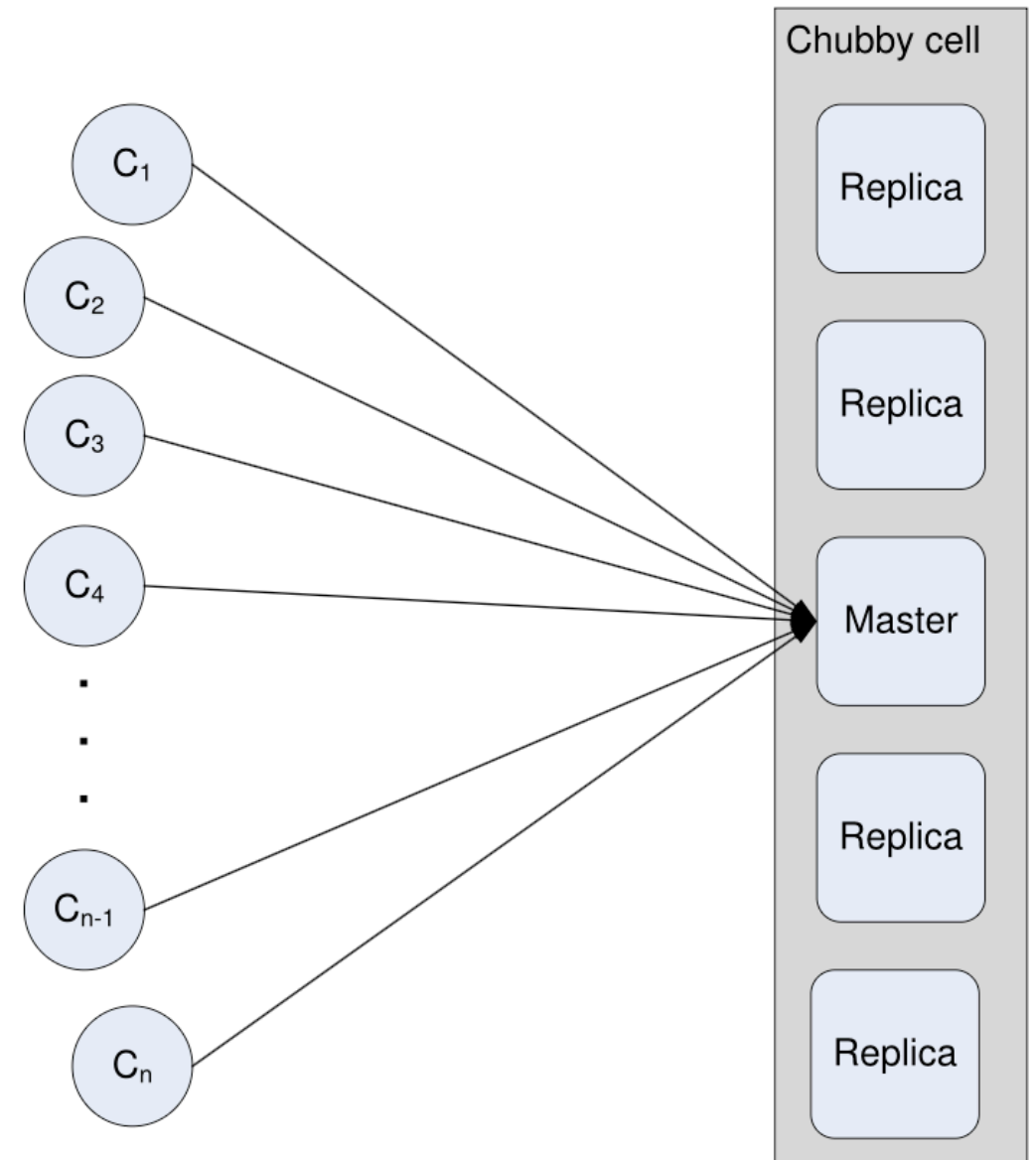
A Chubby cell typically serves one data center.

A cell contains several replicas; the standard number is five.

One replica is elected master.

Clients communicate with the master using RPCs.

When the master receives a write request, it propagates the request to replicas and waits for a majority.

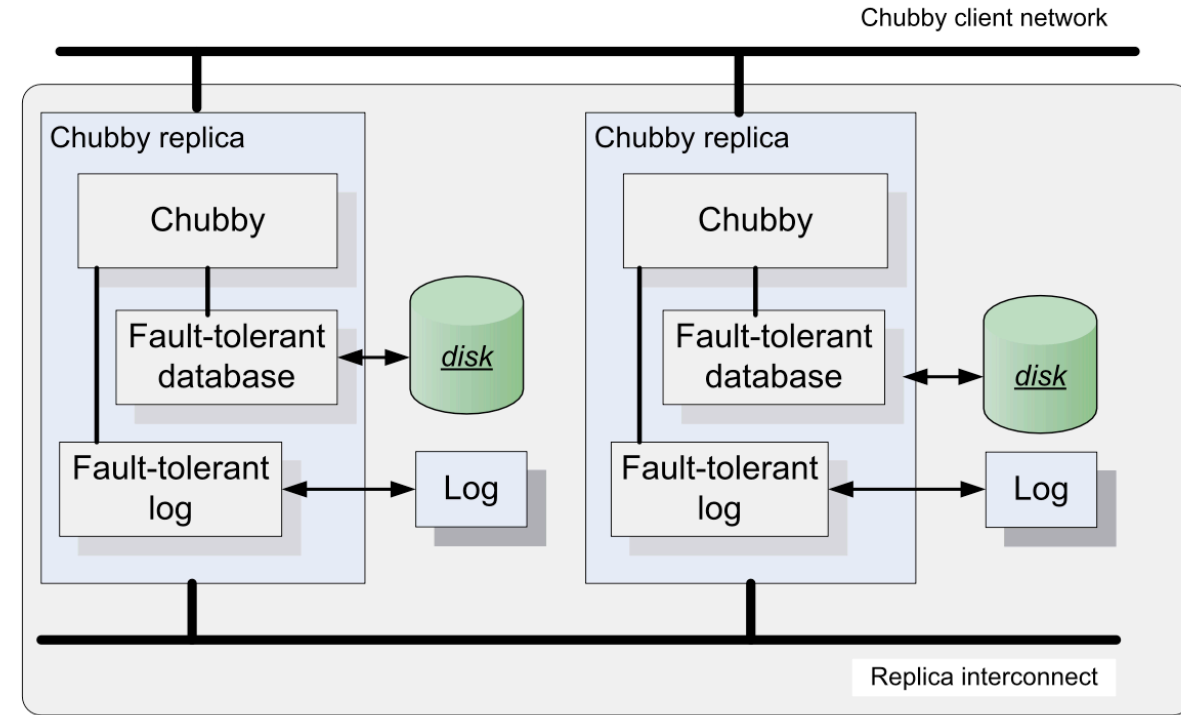


Chubby Architecture

Chubby replicas include:

- Chubby protocol component
- Fault-tolerant database
- Fault-tolerant log
- Paxos consensus support
- Local file system
- Dedicated replica interconnect
- Client network interface

Burrows, M. "The Chubby lock service for loosely-coupled distributed systems." OSDI 2006.



Chubby Usage

Chubby files and directories are organized in a tree structure.

Every file or directory can act as a lock.

Example:

1. Three GFS master candidates (A, B, C) start and connect to the Chubby cell.
2. Each tries to acquire an exclusive lock on `/gfs/master`.
3. Chubby grants the lock to **A** and rejects **B** and **C**.
4. A becomes the active GFS master and periodically renews its lease.
5. B and C watch the lock and remain idle.
6. If A crashes or its lease expires, Chubby releases the lock.
7. B and C compete again; the winner becomes the new master.

Storage System Design Principles

Storage Reliability at Scale

Cloud storage must build reliable systems from unreliable components.

Failures include:

- Disk failures
- Server failures
- Network failures
- Software failures
- Data center failures

The larger the system, the more likely some component is failing at any moment.

Replication

Replication improves:

- Availability
- Durability
- Read throughput
- Fault tolerance

But replication creates challenges:

- Where should replicas be placed?
- How many replicas are enough?
- How are replicas kept consistent?
- How is stale data detected?
- How does the system recover from permanent failures?

Disk Locality vs. Data Locality

Traditionally:

| Run tasks where their input data is stored on disk.

Alternative view:

| Data locality in memory is more important.

Reasons:

- Networks improve rapidly
- Memory is much faster than disk
- Many data blocks are accessed only once
- Cluster memory can cache useful working sets

Locality and Performance

Locality helps because:

1. Local disk bandwidth can be higher than network bandwidth
2. Local disk access has lower latency than remote disk access
3. Off-rack communication may be oversubscribed

Summary

Systems Covered

System	Main idea
NFS	Client-server remote file access
AFS	Cached distributed file system with security focus
Sprite FS	Aggressive client/server caching
GPFS	Parallel file system for large clusters
GFS	Large-chunk distributed file system for commodity clusters
Chubby	Paxos-based distributed lock service
BigTable	Sparse distributed multidimensional table
Megastore	Partitioned, replicated storage with ACID inside partitions